

Semester 01 2025/2026

Subject : Database Programming (SECP3623)

Section : Section 1

Task : Project I

Video : <https://youtu.be/KhOBSrmbwIU>

Name	Matric number
LEE YIN SHEN	A23CS0236
LAU YEE WEN	A23CS0099
POH LOK YEE	A23CS0260
CHERYL CHEONG KAH VOON	A23CS0060

Instructions

Each group must submit **one (1)** project only.

Your submission must include:

1. **Project Report** in **PDF format** (named as LeaderName_GroupNum_ProjectI.pdf).
2. The **demo video URL** (YouTube or Google Drive link) **inside the report**.
 - Make sure the link is **working, publicly accessible**, and does **not require login**.
3. Include all SQL script files (.sql) by attaching them directly, compressing them into one zip folder, or providing a valid OneDrive sharing link inside the report.

Table of content

1.0 Introduction.....	3
1.1 Background of the Chosen Domain (Pet Hospital System).....	3
1.2 Project Objectives.....	3
1.3 Team Members and Roles.....	4
2.0 SQL Implementation Summary.....	4
2.1 SQL File Structure.....	4
2.2 Example SQL Snippets.....	5
2.3 Database Constraints Used.....	7
3.0 Query Demonstrations.....	8
4.0 Optimization Section.....	16
5.0 Team Work.....	17
6.0 Conclusion.....	18

1.0 Introduction

1.1 Background of the Chosen Domain (Pet Hospital System)

The chosen domain for this project is a **Pet Hospital System**, also known as a Veterinary Clinic Management System. This domain is mainly on managing all information related to animal health. A modern pet hospital must process a large number of various interrelated data, including detailed patient records (pet's species, breed, medical history, vaccinations), customer information (owner's name, phone, address) and operation details (vet schedules, appointments). The main challenge is ensuring this information is accurate, secure, and quickly accessible. A comprehensive database system is important for simplifying daily operations, from appointments to retrieving pet medical history in emergency situations, which is important to ensure efficient workflow and high-quality patient care.

1.2 Project Objectives

Our main goal is to build the backend database for this Pet Hospital System using MySQL. This database will be the "brain" that stores and manages all the clinic's information.

Our specific goals are:

1. To design a clean database structure that links owners, pets, vets, appointments, and medical records.
2. To build the tables using SQL (CREATE TABLE) and rules (like PRIMARY KEY and FOREIGN KEY) to protect the data.
3. To add realistic sample data (INSERT) and show how to change it (UPDATE, DELETE).
4. To filter the data using SELECT queries to find useful information (using joins, grouping, and subqueries).
5. To speed up the queries using indexes and show how it works.

1.3 Team Members and Roles

Member Name	Roles
CHERYL CHEONG KAH VOON	Database Creation (DDL)
LEE YIN SHEN	Data Manipulation (DML)
POH LOK YEE	Data Retrieval (DQL / SELECT)
LAU YEE WEN	Indexing and Optimization. Report overall format adjustment and review.

2.0 SQL Implementation Summary

2.1 SQL File Structure

To make sure our project is organized and easy to understand, we divide the SQL code into four main files:

1. **P1_DDL.sql:** This file contains all Data Definition Language (DDL) commands. It is responsible for creating the `pet_hospital_system` database and all five tables, including `Owner`, `Vet`, `Pet`, `Appointment` and `Medical_Record`. It also includes the `ALTER TABLE` statement.
2. **P2_DML.sql:** This file contains all Data Manipulation Language (DML) commands. Its purpose is to use the `INSERT` statement to fill all tables with at least 10 lines of actual sample data. It also includes the `UPDATE` and `DELETE` statements required for demonstration data maintenance.
3. **P3_DQL.sql:** This file contains all the Data Query Language (DQL) statements. It includes all `SELECT` queries we developed to retrieve meaningful information from the database, covering all needs from simple filtering to complex multi-table connections, subqueries and aggregation.
4. **P4_Indexing.sql:** This file contains the `CREATE INDEX` command for the project optimisation part, and the `EXPLAIN` query for analysing the performance before and after the index.
5. **dp_project.sql:** This is the complete source code file for the entire project. It combines all DDL, DML, DQL, and indexing commands into a single script, allowing

the entire database system to be built, populated, and queried in one execution.

2.2 Example SQL Snippets

No	Snippet
1	<pre>CREATE TABLE Appointment (appointment_id INT PRIMARY KEY, pet_id INT NOT NULL, vet_id INT NOT NULL, appointment_date DATE, reason VARCHAR (255), FOREIGN KEY (pet_id) REFERENCES Pet(pet_id), FOREIGN KEY (vet_id) REFERENCES Vet(vet_id));</pre> Table Creation (DDL): This snippet shows the creation of the Appointment table using CREATE TABLE. It is a table that links the Pet and Vet table together using FOREIGN KEY, establishing the core relationship of our system.
2	<pre>INSERT INTO Owner (owner_id, owner_name, phone_num, email, address) VALUES (1, 'Alice Tan', '+6012-1234567', 'alice.tan@example.com', '1, Jalan Pahlawan, 81300 Skudai, Johor'), UPDATE Owner SET address = 'Balai Polis, 81310 Skudai, Johor', phone_num = '+6012-3456789' WHERE owner_id = 1 ;</pre> Data Manipulation (DML): This snippet shows inserting a pet record and updating an owner's address and phone number. We used INSERT to insert data and UPDATE to modify records.

3	<pre> SELECT a.appointment_date, v.vet_name, p.pet_name, o.owner_name, o.phone_num FROM appointment a INNER JOIN vet v ON v.vet_id = a.vet_id INNER JOIN pet p ON p.pet_id = a.pet_id INNER JOIN owner o ON p.owner_id = o.owner_id WHERE a.appointment_date BETWEEN '2025-06-01' AND '2025-11-30' ORDER BY a.appointment_date; </pre> Complex Data Retrieval: This snippet shows how we link four different tables (Appointment, Vet, Pet, Owner) using JOIN to generate a comprehensive appointment report.
4	<pre> SELECT species, COUNT(pet_id) AS Total, SUM(weight) AS Total_Weight, ROUND(AVG(weight), 2) AS Average_Weight, MIN(weight) AS Min_Weight, MAX(weight) AS Max_Weight FROM pet GROUP BY species; </pre> Aggregation and Grouping: This snippet shows how we calculate statistics for each species using COUNT, SUM, ROUND, MIN and MAX, showing the total number of pets and their average weight.
5	<pre> CREATE INDEX idx_appt_date ON Appointment(appointment_date); EXPLAIN SELECT * FROM appointment WHERE appointment_date BETWEEN '2025-01-01' AND '2025-06-30'; </pre> B-Tree Index: This snippet shows how we create a B-Tree index on the appointment_date column using CREATE INDEX, and prove the use of index using EXPLAIN. This is important for optimizing the range queries

6

```
CREATE FULLTEXT INDEX idx_medical_text
ON Medical_Record(diagnosis, treatment, note);

EXPLAIN SELECT * FROM Medical_Record
WHERE MATCH(diagnosis, treatment, note) AGAINST('vaccination');
```

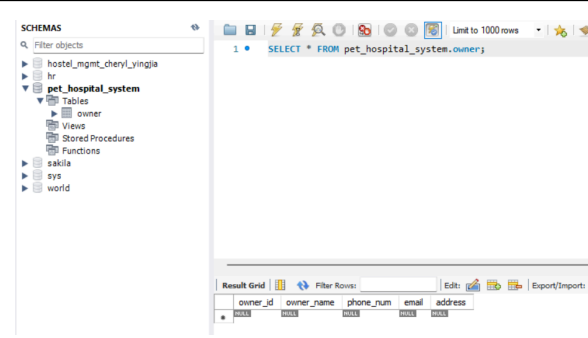
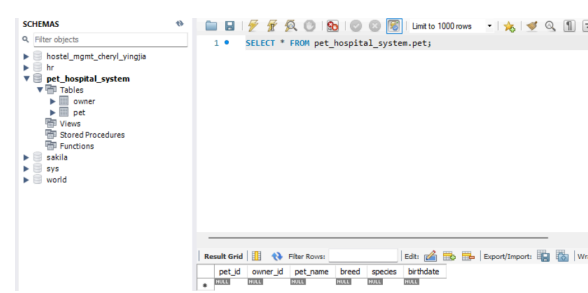
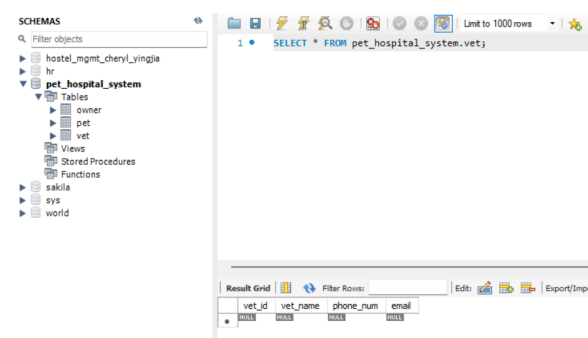
Full-Text Index: This snippet shows how we create a full-text index on the `appointment_date` column using `CREATE FULLTEXT INDEX`, and prove the use of index using `EXPLAIN`. This allows the user to quickly find records based on keywords in diagnoses or notes.

2.3 Database Constraints Used

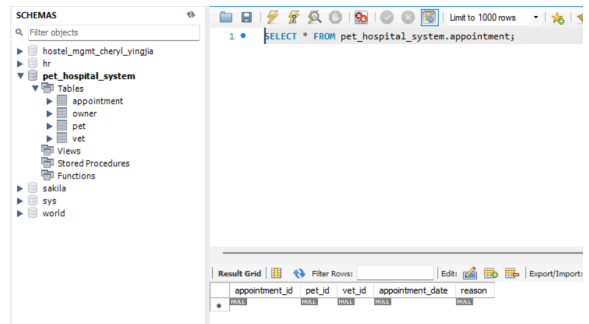
We used several types of constraints to ensure data integrity and accuracy in the pet hospital.

1. **PRIMARY KEY:** Used on ID columns like `owner_id`, `pet_id` and `vet_id` to uniquely identify every row in a table. This is important to link between tables and prevent duplicate entries.
2. **FOREIGN KEY:** Used to create links between tables and enforce referential integrity. For example, `Pet.owner_id` references to `Owner.owner_id`, which ensures that a pet cannot be added to the database without a valid owner.
3. **NOT NULL:** Applied to essential columns like `owner_name` and `pet_name` to ensure that this critical information is not blank.
4. **UNIQUE:** Used on columns like `Owner.phone_num` and `Owner.email` to prevent two owners from having the same data, which are unique identifiers in the real world.

3.0 Query Demonstrations

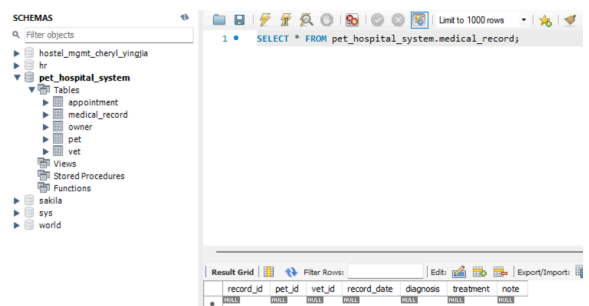
SQL Query	Output / Result
A. Database Creation (DDL)	
Create the Owner table using CREATE TABLE	
<pre>CREATE TABLE Owner (owner_id INT PRIMARY KEY, owner_name VARCHAR (255) NOT NULL, phone_num VARCHAR (255) UNIQUE, email VARCHAR (200) UNIQUE, address VARCHAR (255));</pre>	
Create the Pet table using CREATE TABLE with a foreign key referencing the Owner table	
<pre>CREATE TABLE Pet (pet_id INT PRIMARY KEY, owner_id INT NOT NULL, pet_name VARCHAR (255) NOT NULL, breed VARCHAR (100) NOT NULL, species VARCHAR(50) NOT NULL, birthdate DATE, FOREIGN KEY (owner_id) REFERENCES Owner(owner_id));</pre>	
Create the Vet table using CREATE TABLE	
<pre>CREATE TABLE Vet (vet_id INT PRIMARY KEY, vet_name VARCHAR (255) NOT NULL, phone_num VARCHAR (255) UNIQUE, email VARCHAR (200) UNIQUE);</pre>	
Create the Appointment table using CREATE TABLE with a foreign key referencing the Pet table and the Vet table	

```
CREATE TABLE Appointment (
    appointment_id INT PRIMARY KEY,
    pet_id INT NOT NULL,
    vet_id INT NOT NULL,
    appointment_date DATE,
    reason VARCHAR (255),
    FOREIGN KEY (pet_id) REFERENCES Pet(pet_id),
    FOREIGN KEY (vet_id) REFERENCES Vet(vet_id)
);
```



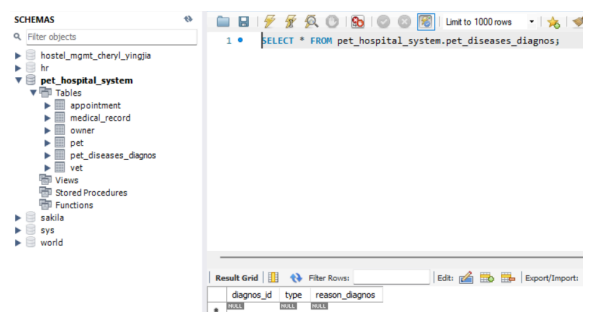
Create the Medical_Record table using CREATE TABLE with a foreign key referencing the Pet table and Vet table

```
CREATE TABLE Medical_Record (
    record_id INT PRIMARY KEY,
    pet_id INT NOT NULL,
    vet_id INT NOT NULL,
    record_date DATE,
    diagnosis VARCHAR (255),
    treatment VARCHAR (255),
    note TEXT,
    FOREIGN KEY (pet_id) REFERENCES Pet(pet_id),
    FOREIGN KEY (vet_id) REFERENCES Vet(vet_id)
);
```



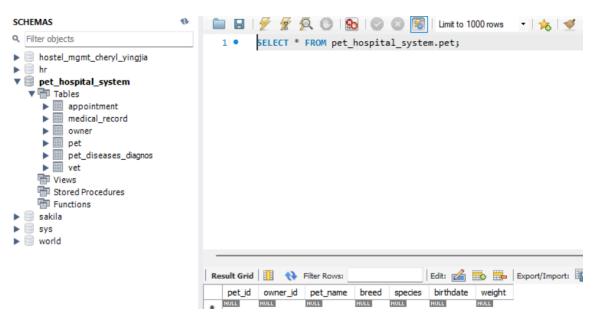
Create the Pet_diseases_diagnos table using CREATE TABLE

```
CREATE TABLE Pet_diseases_diagnos (
    diagnos_id INT PRIMARY KEY,
    type VARCHAR (255),
    reason_diagnos VARCHAR (255)
);
```



Modify the Pet table structure using ALTER TABLE to add the weight column

```
ALTER TABLE Pet
ADD COLUMN weight DECIMAL (4,2);
```



Drop the Pet_diseases_diagnos table safely using DROP TABLE IF EXISTS

```
DROP TABLE IF EXISTS Pet_diseases_diagnos;
```

The screenshot shows the SQL Editor interface with the 'pet_hospital_system' database selected. The SQL code in the editor is as follows:

```

42      pet_id INT NOT NULL,
43      vet_id INT NOT NULL,
44      record_date DATE,
45      diagnosis VARCHAR (255),
46      treatment VARCHAR (255),
47      note TEXT,
48      FOREIGN KEY (pet_id) REFERENCES Pet(pet_id),
49      FOREIGN KEY (vet_id) REFERENCES Vet(vet_id),
50  };
51
52  ● CREATE TABLE Pet_diseases_diagnos (
53      diagnosis_id INT PRIMARY KEY,
54      type VARCHAR (255),
55      reason_diagnos VARCHAR (255)
56  );
57
58
59  ● ALTER TABLE Pet
60      ADD COLUMN weight DECIMAL (4,2);
61
62  ● DROP TABLE IF EXISTS Pet_diseases_diagnos;

```

B. Data Manipulation (DML)

Insert data into Owner table using INSERT INTO ... VALUES.

```
INSERT INTO Owner (owner_id, owner_name, phone_num, email, address) VALUES
(1, 'Alice Tan', '+6012-1234567', 'alice.tan@example.com', '1, Jalan Pahlawan,
(2, 'Bob Lee', '+6013-2345678', 'bob.lee@example.com', '2, Jalan Merdeka, 80101
(3, 'Charlie Lim', '+6014-3456789', 'charlie.lim@example.com', '3, Jalan Setia
(4, 'David Chen', '+6016-4567890', 'david.chen@example.com', '4, Jalan Indah,
(5, 'Emily Yong', '+6017-5678901', 'emily.yong@example.com', '5, Jalan Harmoni
(6, 'Faridah Hassan', '+6018-6789012', 'faridah.h@example.com', '6, Jalan Besti
(7, 'Ganesh Raj', '+6019-7890123', 'ganesh.raj@example.com', '7, Jalan Mutiara
(8, 'Helen Woo', '+6011-8901234', 'helen.woo@example.com', '8, Jalan Sasa, 8171
(9, 'Idris bin Saleh', '+6010-9012345', 'idris.saleh@example.com', '9, Jalan Ceri
(10, 'Jasmine Kaur', '+6015-0123456', 'jasmine.k@example.com', '10, Jalan Perw
```

owner_id	owner_name	phone_num	email	address
1	Alice Tan	+6012-1234567	alice.tan@example.com	1, Jalan Pahlawan, 81300 Skudai, Johor
2	Bob Lee	+6013-2345678	bob.lee@example.com	1, Jalan Merdeka, 80100 Johor Bahru, Johor
3	Charlie Lim	+6014-3456789	charlie.lim@example.com	3, Jalan Setia, 79100 Iskandar Puteri, Johor
4	David Chen	+6016-4567890	david.chen@example.com	1, Jalan Indah, 81100 Masai, Johor
5	Emily Tong	+6015-5678901	emily.yong@example.com	1, Jalan Harmoni, 81750 Ponteng, Johor
6	Faridhan Hassan	+6018-6789012	faridah.h@example.com	1, Jalan Bestari, 81300 Skudai, Johor
7	Ganesh Raj	+6019-7890123	ganesh.raj@example.com	1, Jalan Mutiara, 81100 Johor Bahru, Johor
8	Helen Woo	+6011-8901234	helen.woo@example.com	1, Jalan Setia, 81700 Pengerang, Johor
9	Irfan bin Saleh	+6019-9012345	idris.saleh@example.com	1, Jalan CERIA, 81300 Skudai, Johor
10	Jasmine Kaur	+6015-123456	jasmine.k@example.com	1, Jalan Pervira, 80400 Johor Bahru, Johor

Insert data into Vet table using INSERT INTO ... VALUES.

```
INSERT INTO Vet (vet_id, vet_name, phone_num, email) VALUES
(1, 'Dr. Sarah Chen', '+6012-1112222', 'sarah.chen@gmail.com'),
(2, 'Dr. Kumar Moorthy', '+6012-3334444', 'kumar.moorthy@gmail.com'),
(3, 'Dr. Aisha Ibrahim', '+6012-5556666', 'aisha.ibrahim@gmail.com'),
(4, 'Dr. Benjamin Low', '+6012-7778888', 'ben.low@gmail.com'),
(5, 'Dr. Felicia Ong', '+6012-9990000', 'felicia.ong@gmail.com'),
(6, 'Dr. Michael Tan', '+6013-1112222', 'michael.tan@gmail.com'),
(7, 'Dr. Rachel Lim', '+6013-3334444', 'rachel.lim@gmail.com'),
(8, 'Dr. Samuel David', '+6013-5556666', 'samuel.david@gmail.com'),
(9, 'Dr. Siti Nurhaliza', '+6013-7778888', 'siti.nurhaliza@gmail.com'),
(10, 'Dr. Ivan Petrov', '+6013-9990000', 'ivan.petrov@gmail.com');
```

vet_id	vet_name	phone_num	email
1	Dr. Sarah Chen	+6012-1112222	sarah.chen@gmail.com
2	Dr. Kumar Moorthy	+6012-3334444	kumar.moorthy@gmail.com
3	Dr. Aisha Ibrahim	+6012-5556666	aisha.ibrahim@gmail.com
4	Dr. Benjamin Low	+6012-7778888	ben.low@gmail.com
5	Dr. Felicia Ong	+6012-9990000	felicia.ong@gmail.com
6	Dr. Michael Tan	+6013-1112222	michael.tan@gmail.com
7	Dr. Rachel Lim	+6013-3334444	rachel.lim@gmail.com
8	Dr. Samuel David	+6013-5556666	samuel.david@gmail.com
9	Dr. Siti Nurhaliza	+6013-7778888	siti.nurhaliza@gmail.com
10	Dr. Ivan Petrov	+6013-9990000	ivan.petrov@gmail.com

Insert data into Pet table using INSERT INTO ... VALUES.

```
INSERT INTO Pet (pet_id, owner_id, pet_name, breed, species, birthdate, weight)
VALUES
(1, 1, 'Max', 'Golden Retriever', 'Dog', '2020-05-10', 30.50),
(2, 1, 'Whiskers', 'Siamese', 'Cat', '2019-01-15', 4.20),
(3, 2, 'Buddy', 'Labrador', 'Dog', '2022-03-01', 28.00),
(4, 3, 'Fluffy', 'Persian', 'Cat', '2021-11-20', 3.80),
(5, 4, 'Rocky', 'German Shepherd', 'Dog', '2018-07-30', 35.10),
(6, 5, 'Mittens', 'Domestic Shorthair', 'Cat', '2023-01-05', 3.00),
(7, 5, 'Squeaky', 'Syrian', 'Rodent', '2024-02-14', 0.15),
(8, 6, 'Goldie', 'Goldfish', 'Fish', '2023-05-10', 0.05),
(9, 7, 'Charlie', 'Poodle', 'Dog', '2021-06-12', 12.50),
(10, 8, 'Shadow', 'Bombay', 'Cat', '2020-08-08', 4.50),
(11, 8, 'Twenty', 'Canary', 'Bird', '2022-12-01', 0.10),
(12, 9, 'Spike', 'Bulldog', 'Dog', '2019-10-10', 24.00),
(13, 10, 'Luna', 'Maine Coon', 'Cat', '2021-02-28', 6.80),
(14, 1, 'Bella', 'Beagle', 'Dog', '2023-04-15', 10.20),
(15, 2, 'Nibbles', 'Log-eared', 'Rabbit', '2022-11-01', 2.10);
```

pet_id	owner_id	pet_name	breed	species	birthdate	weight
1	1	Max	Golden Retriever	Dog	2020-05-10	30.50
2	1	Whiskers	Siamese	Cat	2019-01-15	4.20
3	2	Buddy	Labrador	Dog	2022-03-01	28.00
4	3	Fluffy	Persian	Cat	2021-11-20	3.80
5	4	Rocky	German Shepherd	Dog	2018-07-30	35.10
6	5	Mittens	Domestic Shorthair	Cat	2023-01-05	3.00
7	5	Squeaky	Syrian	Rodent	2024-02-14	0.15
8	6	Goldie	Goldfish	Fish	2023-05-10	0.05
9	7	Charlie	Poodle	Dog	2021-06-12	12.50
10	8	Shadow	Bombay	Cat	2020-08-08	4.50
11	8	Tweety	Canary	Bird	2022-12-01	0.10
12	9	Spike	Bulldog	Dog	2019-10-10	24.00
13	10	Luna	Maine Coon	Cat	2021-02-28	6.80
14	1	Bella	Beagle	Dog	2023-04-15	10.20
15	2	Nibbles	Long-eared	Rabbit	2022-11-01	2.10

Insert data into Appointment table using INSERT INTO ... VALUES.

<pre>INSERT INTO Appointment (appointment_id, pet_id, vet_id, appointment_date, reason) VALUES (1, 1, 1, '2024-12-05', 'Annual Vaccination'), (2, 2, 2, '2024-12-18', 'Skin rash check'), (3, 3, 3, '2025-01-10', 'Limping'), (4, 4, 4, '2025-01-25', 'Dental check'), (5, 1, 1, '2025-02-05', 'Follow-up on vaccination'), (6, 5, 5, '2025-02-20', 'General checkup'), (7, 7, 6, '2025-03-12', 'Not eating'), (8, 9, 7, '2025-03-28', 'Coughing'), (9, 10, 8, '2025-04-15', 'Checkup'), (10, 12, 9, '2025-05-02', 'Breathing issue'), (11, 13, 10, '2025-05-30', 'Grooming query'), (12, 14, 1, '2025-06-15', 'First puppy shots'), (13, 2, 2, '2025-07-01', 'Skin rash follow-up'), (14, 6, 3, '2025-07-22', 'Spaying procedure query'), (15, 3, 4, '2025-08-10', 'Limping follow-up'), (16, 8, 5, '2025-09-05', 'Fungus on scales'), (17, 11, 6, '2025-09-25', 'Beak trimming'), (18, 15, 7, '2025-10-15', 'Nail clipping'), (19, 5, 8, '2025-11-01', 'Senior dog checkup'), (20, 1, 9, '2025-11-15', 'Eating grass');</pre>				
appointment_id	pet_id	vet_id	appointment_date	reason
1	1	1	2024-12-05	Annual Vaccination
2	2	2	2024-12-18	Skin rash check
3	3	3	2025-01-10	Limping
4	4	4	2025-01-25	Dental check
5	1	1	2025-02-05	Follow-up on vaccination
6	5	5	2025-02-20	General checkup
7	7	6	2025-03-12	Not eating
8	9	7	2025-03-28	Coughing
9	10	8	2025-04-15	Checkup
10	12	9	2025-05-02	Breathing issue
11	13	10	2025-05-30	Grooming query
12	14	1	2025-06-15	First puppy shots
13	2	2	2025-07-01	Skin rash follow-up
14	6	3	2025-07-22	Spaying procedure query
15	3	4	2025-08-10	Limping follow-up
16	8	5	2025-09-05	Fungus on scales
17	11	6	2025-09-25	Beak trimming
18	15	7	2025-10-15	Nail clipping
19	5	8	2025-11-01	Senior dog checkup
20	1	9	2025-11-15	Eating grass
NULL	NULL	NULL	NULL	NULL

Insert data into Medical_Record table using INSERT INTO ... VALUES.

INSERT INTO Medical_Record (record_id, pet_id, vet_id, record_date, diagnosis, treatment, note) VALUES						
(1, 1, 1, '2024-12-05', 'N/A (Vaccination)', 'Administered DAPPv, Rabies vaccine.', 'Pet healthy. N vaccine.')						
(2, 2, 2, '2024-12-18', 'Allergic Dermatitis', 'Prescribed antihistamine, medicated shampoo.', 'Follow-up in 4 weeks.')						
(3, 3, 3, '2025-01-10', 'Minor Sprain (Left Hind Leg)', 'Prescribed rest and anti-inflammatory meds.', 'Re-check if no improvement.')						
(4, 4, 4, '2025-01-25', 'Mild Tartar Buildup', 'Dental cleaning procedure performed.', 'Advised owner brushing.')						
(5, 1, 1, '2025-02-05', 'Post-Vaccination Check', 'No adverse reactions noted from vaccine.', 'Pet is clear. Next vaccine due in 3 months.')						
(6, 5, 5, '2025-02-20', 'Healthy', 'Full blood panel normal. All clear.', 'Advised weight management.')						
(7, 7, 6, '2025-03-12', 'Dental Malocclusion', 'Trimmed overgrown incisors.', 'Owner advised regular dental checkups.')						
(8, 9, 7, '2025-03-28', 'Kennel Cough (Mild)', 'Prescribed antibiotics and cough suppressant.', 'Quarantine from other dogs.')						
(9, 10, 8, '2025-04-15', 'Healthy', 'Annual checkup, all clear.', 'N/A')						
(10, 12, 9, '2025-05-02', 'Brachycephalic Airway Syndrome', 'Discussed surgical options with owner.', 'Owner to consult with specialist.')						
(11, 13, 10, '2025-05-30', 'Matted Fur', 'Medical grooming, removed severe mats.', 'Owner advised regular grooming.')						
(12, 14, 1, '2025-06-15', 'First Vaccination Series', 'Administered first DAPPv shot.', 'Booster due in 3 months.')						
(13, 2, 2, '2025-07-01', 'Dermatitis improving', 'Continued antihistamines.', 'Swelling reduced.')						
(14, 6, 3, '2025-07-22', 'Pre-Spay Consultation', 'Discussed procedure and risks.', 'Scheduled spay surgery.')						
(15, 3, 4, '2025-08-10', 'Sprain Healed', 'Pet no longer limping.', 'Cleared for normal activity.')						

record_id	pet_id	vet_id	record_date	diagnosis	treatment	note
1	1	1	2024-12-05	N/A (Vaccination)	Administered DAPPv, Rabies vaccine.	Pet healthy. N vaccine.
2	2	2	2024-12-18	Allergic Dermatitis	Prescribed antihistamine, medicated shampoo.	Follow-up in 4 weeks.
3	3	3	2025-01-10	Minor Sprain (Left Hind Leg)	Prescribed rest and anti-inflammatory meds.	Re-check if no improvement.
4	4	4	2025-01-25	Mild Tartar Buildup	Dental cleaning procedure performed.	Advised owner brushing.
5	1	1	2025-02-05	Post-Vaccination Check	No adverse reactions noted from vaccine.	Pet is clear. Next vaccine due in 3 months.
6	5	5	2025-02-20	Healthy	Full blood panel normal. All clear.	Advised weight management.
7	7	6	2025-03-12	Dental Malocclusion	Trimmed overgrown incisors.	Owner advised regular dental checkups.
8	9	7	2025-03-28	Kennel Cough (Mild)	Prescribed antibiotics and cough suppressant.	Quarantine from other dogs.
9	10	8	2025-04-15	Healthy	Annual checkup, all clear.	N/A
10	12	9	2025-05-02	Brachycephalic Airway Syndrome	Discussed surgical options with owner.	Owner to consult with specialist.
11	13	10	2025-05-30	Matted Fur	Medical grooming, removed severe mats.	Owner advised regular grooming.
12	14	1	2025-06-15	First Vaccination Series	Administered first DAPPv shot.	Booster due in 3 months.
13	2	2	2025-07-01	Dermatitis improving	Continued antihistamines.	Swelling reduced.
14	6	3	2025-07-22	Pre-Spay Consultation	Discussed procedure and risks.	Scheduled spay surgery.

Update the Owner and Pet data using UPDATE ... SET ... WHERE.

```

UPDATE Owner
SET address = 'Balai Polis, 81310 Skudai, Johor',
    phone_num = '+6012-3456789'
WHERE owner_id = 1 ;

UPDATE Pet
SET weight = 31.20
WHERE pet_id = 1 ;

```

owner_id	owner_name	phone_num	email	address
1	Alice Tan	+6012-3456789	alice.tan@example.com	Balai Polis, 81310 Skudai, Johor

pet_id	owner_id	pet_name	breed	species	birthdate	weight
1	1	Max	Golden Retriever	Dog	2020-05-10	31.20

Delete the Medical and Appointment data using DELETE FROM ... WHERE

```
DELETE FROM Medical_Record
WHERE record_id = 15 ;

DELETE FROM Appointment
WHERE appointment_id = 7 ;
```

13	4	4	2025-07-01	Dermatitis improving	Continued antihistamines.	Swelling reduced.
14	6	3	2025-07-22	Pre-Spay Consultation	Discussed procedure and risks.	Scheduled spay surgery.
6	5	5	2025-02-20	General checkup		
8	9	7	2025-03-28	Coughing		

Delete all records from a specific Pet using DELETE FROM ... WHERE. Instead of directly deleting a specific pet record from the Pet table, all the associated records from other tables like Medical_Record and Appointment must be deleted first to avoid foreign key error.

```
DELETE FROM Medical_Record
WHERE pet_id = 8;
```

```
DELETE FROM Appointment
WHERE pet_id = 8;
```

```
DELETE FROM Pet
WHERE pet_id = 8;
```

pet_id	owner_id	pet_name	breed	species	birthdate	weight
6	5	Mittens	Domestic Shorthair	Cat	2023-01-05	3.00
7	5	Squeaky	Syrian	Rodent	2024-02-14	0.15
9	7	Charlie	Poodle	Dog	2021-06-12	12.50

C. Data Retrieval (DQL / SELECT)

1. Filtering - Filtering the pet table using SELECT...FROM to retrieve the species that are Dogs with weights between 27 to 45 kg using WHERE....AND.... BETWEEN.

```
# 1.Filtering
SELECT pet_id, pet_name, species, breed, weight
FROM pet
WHERE species = 'Dog' AND weight BETWEEN 27.00 AND 45.00;
```

	pet_id	pet_name	species	breed	weight
▶	1	Max	Dog	Golden Retriever	31.20
	3	Buddy	Dog	Labrador	28.00
	5	Rocky	Dog	German Shepherd	35.10

2. Sorting - Selecting the pet columns then sorting the result by birthdate using ORDER BY, and limiting to 8 rows by using LIMIT.

```
# 2.Sorting
SELECT pet_name, species, birthdate
FROM pet
ORDER BY birthdate LIMIT 8;
```

	pet_name	species	birthdate
▶	Rocky	Dog	2018-07-30
	Whiskers	Cat	2019-01-15
	Spike	Dog	2019-10-10
	Max	Dog	2020-05-10
	Shadow	Cat	2020-08-08
	Luna	Cat	2021-02-28
	Charlie	Dog	2021-06-12
	Fluffy	Cat	2021-11-20

3. Aggregation - Grouping the pet table by species to calculate the COUNT, SUM, AVG, MIN and MAX weight for each group species by using GROUP BY and aggregate function. ROUND are used to round up the value to 2 decimal places.

```
# 3.Aggregation (+ Grouping)
SELECT species,
COUNT(pet_id) AS Total,
SUM(weight) AS Total_Weight,
ROUND(AVG(weight), 2) AS Average_Weight,
MIN(weight) AS Min_Weight,
MAX(weight) AS Max_Weight
FROM pet
GROUP BY species;
```

	species	Total	Total_Weight	Average_Weight	Min_Weight	Max_Weight
▶	Dog	6	141.00	23.50	10.20	35.10
	Cat	5	22.30	4.46	3.00	6.80
	Rodent	1	0.15	0.15	0.15	0.15
	Fish	1	0.05	0.05	0.05	0.05
	Bird	1	0.10	0.10	0.10	0.10
	Rabbit	1	2.10	2.10	2.10	2.10

4. Group Filtering - Grouping appointments by vet_id by using GROUP BY, then filtering those groups to show vet_id with more than 2 appointments by using HAVING and

COUNT.

4. Grouping and filtering groups

```
SELECT vet_id,
       COUNT(appointment_id) AS Total_Appointments
FROM appointment
GROUP BY vet_id
HAVING COUNT(appointment_id) > 2;
```

	vet_id	Total_Appointments
▶	1	3

5. Numeric and String Function - Calculating the LENGTH of pet_name and constructing a new Pet_Code by using CONCAT, UPPER, and SUBSTR. The result is then sorted by Pet_Code by using ORDER BY and limited to 8 rows by using LIMIT.

5. Numeric and string functions

```
SELECT pet_id, pet_name,
       LENGTH(pet_name) AS Name_Length,
       species,
       CONCAT(UPPER(SUBSTR(species, 1, 3)), '-', pet_id) AS Pet_Code
FROM pet
ORDER BY Pet_Code LIMIT 8;
```

	pet_id	pet_name	Name_Length	species	Pet_Code
▶	11	Tweety	6	Bird	BIR-11
	10	Shadow	6	Cat	CAT-10
	13	Luna	4	Cat	CAT-13
	2	Whiskers	8	Cat	CAT-2
	4	Fluffy	6	Cat	CAT-4
	6	Mittens	7	Cat	CAT-6
	1	Max	3	Dog	DOG-1
	12	Spike	5	Dog	DOG-12

6. Conditional Logic - Categorizing the weight column into size classes 'Giant', 'Large', 'Medium' and 'Small' for Dog species by using CASE WHEN.

6. Conditional logic (CASE WHEN).

```
SELECT pet_name, breed, weight,
       CASE
       WHEN weight >= 45 THEN 'Giant'
       WHEN weight >= 27 THEN 'Large'
       WHEN weight >= 11 THEN 'Medium'
       ELSE 'Small'
       END AS Size
FROM pet
WHERE species = 'DOG';
```

	pet_name	breed	weight	Size
▶	Max	Golden Retriever	31.20	Large
	Buddy	Labrador	28.00	Large
	Rocky	German Shepherd	35.10	Large
	Charlie	Poodle	12.50	Medium
	Spike	Bulldog	24.00	Medium
	Bella	Beagle	10.20	Small

7. Subqueries (Single-row) - Filtering the pet table by weight that equals the result of an inner query by using subquery SELECT...FROM in WHERE...=.

7. Subqueries

Single-row

```
SELECT pet_name
FROM pet
WHERE weight = (
    SELECT MAX(weight)
    FROM pet
);
```

	pet_name
▶	Rocky

7. Subqueries (Multiple-row) - Filtering the owner table by owner_id to find a list that owner has a cat with more than 4kg weight by using subquery SELECT... FROM... WHERE... in WHERE...IN.

```
# multiple_row
SELECT owner_name
FROM owner
WHERE owner_id IN (
    SELECT owner_id
    FROM Pet
    WHERE species = 'Cat' AND weight >= 4
);
```

	owner_name
▶	Alice Tan
	Helen Woo
	Jasmine Kaur

8. Set Operation (UNION) - Combining the owner and vet tables into a single list by using UNION then ordering the combined result by using ORDER BY.

```
# 8.Set Operation
#UNION
SELECT owner_name AS Name, 'Owner' AS Role, phone_num, email
FROM owner
UNION (
    SELECT vet_name AS Vet_Name, 'Vet' AS Role, phone_num, email
    FROM vet)
ORDER BY Role;
```

	Name	Role	phone_num	email
▶	Alice Tan	Owner	+6012-3456789	alice.tan@example.com
	Bob Lee	Owner	+6013-2345678	bob.lee@example.com
	Charlie Lim	Owner	+6014-3456789	charlie.lim@example.com
	David Chen	Owner	+6016-4567890	david.chen@example.com
	Emily Yong	Owner	+6017-5678901	emily.yong@example.com
	Faridah Hassan	Owner	+6018-6789012	faridah.h@example.com
	Ganesh Raj	Owner	+6019-7890123	ganesh.raj@example.com
	Helen Woo	Owner	+6011-8901234	helen.woo@example.com
	Idris bin Saleh	Owner	+6010-9012345	idris.saleh@example.com
	Jasmine Kaur	Owner	+6015-0123456	jasmine.k@example.com
	Dr. Sarah Chen	Vet	+6012-1112222	sarah.chen@gmail.com
	Dr. Kumar Moo...	Vet	+6012-3334444	kumar.moorthy@gmail.com
	Dr. Aisha Ibrahim	Vet	+6012-5556666	aisha.ibrahim@gmail.com
	Dr. Benjamin Low	Vet	+6012-7778888	ben.low@gmail.com
	Dr. Felicia Ong	Vet	+6012-9990000	felicia.ong@gmail.com
	Dr. Michael Tan	Vet	+6013-1112222	michael.tan@gmail.com
	Dr. Rachel Lim	Vet	+6013-3334444	rachel.lim@gmail.com
	Dr. Samuel David	Vet	+6013-5556666	samuel.david@gmail.com
	Dr. Siti Nurhaliza	Vet	+6013-7778888	siti.nurhaliza@gmail.com
	Dr. Ivan Petrov	Vet	+6013-9990000	ivan.petrov@gmail.com

8. Set Operation (NOT EXISTS) - Filtering the pet table to find pets that do not have a medical record with 'Healthy' diagnosis by using WHERE NOT EXISTS.

<pre>#NOT EXISTS(correlated) SELECT p.pet_name, p.species FROM pet p WHERE NOT EXISTS(SELECT * FROM medical_record m WHERE m.pet_id = p.pet_id AND m.diagnosis = 'Healthy');</pre>	<table border="1"> <thead> <tr> <th>pet_name</th><th>species</th></tr> </thead> <tbody> <tr><td>Max</td><td>Dog</td></tr> <tr><td>Whiskers</td><td>Cat</td></tr> <tr><td>Buddy</td><td>Dog</td></tr> <tr><td>Fluffy</td><td>Cat</td></tr> <tr><td>Mittens</td><td>Cat</td></tr> <tr><td>Squeaky</td><td>Rodent</td></tr> <tr><td>Goldie</td><td>Fish</td></tr> <tr><td>Charlie</td><td>Dog</td></tr> <tr><td>Tweety</td><td>Bird</td></tr> <tr><td>Spike</td><td>Dog</td></tr> <tr><td>Luna</td><td>Cat</td></tr> <tr><td>Bella</td><td>Dog</td></tr> <tr><td>Nibbles</td><td>Rabbit</td></tr> </tbody> </table>	pet_name	species	Max	Dog	Whiskers	Cat	Buddy	Dog	Fluffy	Cat	Mittens	Cat	Squeaky	Rodent	Goldie	Fish	Charlie	Dog	Tweety	Bird	Spike	Dog	Luna	Cat	Bella	Dog	Nibbles	Rabbit
pet_name	species																												
Max	Dog																												
Whiskers	Cat																												
Buddy	Dog																												
Fluffy	Cat																												
Mittens	Cat																												
Squeaky	Rodent																												
Goldie	Fish																												
Charlie	Dog																												
Tweety	Bird																												
Spike	Dog																												
Luna	Cat																												
Bella	Dog																												
Nibbles	Rabbit																												

9. Joins (INNER JOIN) - Combining appointment, vet, pet and owner tables based on matching ID keys by using INNER JOIN and filtering the result with a date range by using WHERE....BETWEEN...AND, the result then ordering with appointment date by using ORDER BY. The result displays the appointment date, the corresponding vet, the pet, and its owner and the contact number of the owner.

```
# 9.Joins
SELECT a.appointment_date, v.vet_name, p.pet_name, o.owner_name, o.phone_num
FROM appointment a
INNER JOIN vet v ON v.vet_id = a.vet_id
INNER JOIN pet p ON p.pet_id = a.pet_id
INNER JOIN owner o ON o.owner_id = a.owner_id
WHERE a.appointment_date BETWEEN '2025-06-01' AND '2025-11-30'
ORDER BY a.appointment_date;
```

	appointment_date	vet_name	pet_name	owner_name	phone_num
▶	2025-06-15	Dr. Sarah Chen	Bella	Alice Tan	+6012-3456789
	2025-07-01	Dr. Kumar Moorthy	Whiskers	Alice Tan	+6012-3456789
	2025-07-22	Dr. Aisha Ibrahim	Mittens	Emily Yong	+6017-5678901
	2025-08-10	Dr. Benjamin Low	Buddy	Bob Lee	+6013-2345678
	2025-09-05	Dr. Felicia Ong	Goldie	Faridah Hassan	+6018-6789012
	2025-09-25	Dr. Michael Tan	Tweety	Helen Woo	+6011-8901234
	2025-10-15	Dr. Rachel Lim	Nibbles	Bob Lee	+6013-2345678
	2025-11-01	Dr. Samuel David	Rocky	David Chen	+6016-4567890
	2025-11-15	Dr. Siti Nurhaliza	Max	Alice Tan	+6012-3456789

9. Joins (LEFT JOIN) - Combining the owner table with pet and medical_record table to show all owners are listed even if their pets have no medical records by using LEFT JOIN and then ordering the result with record_id by using ORDER BY.

```
SELECT o.owner_name, p.pet_name, p.species, p.weight, m.diagnosis, m.record_date
FROM owner o
LEFT JOIN pet p ON p.owner_id = o.owner_id
LEFT JOIN medical_record m ON m.pet_id = p.pet_id
ORDER BY m.record_id;
```

owner_name	pet_name	species	weight	diagnosis	record_date
Bob Lee	Nibbles	Rabbit	2.10	NULL	NULL
Faridah Hassan	Goldie	Fish	0.05	NULL	NULL
Helen Woo	Tweety	Bird	0.10	NULL	NULL
Alice Tan	Max	Dog	31.20	N/A (Vaccination)	2024-12-05
Alice Tan	Whiskers	Cat	4.20	Allergic Dermatitis	2024-12-18
Bob Lee	Buddy	Dog	28.00	Minor Sprain (Left Hind Leg)	2025-01-10
Charlie Lim	Fluffy	Cat	3.80	Mild Tartar Buildup	2025-01-25
Alice Tan	Max	Dog	31.20	Post-Vaccination Check	2025-02-05
David Chen	Rocky	Dog	35.10	Healthy	2025-02-20
Emily Yong	Squeaky	Rodent	0.15	Dental Malocclusion	2025-03-12
Ganesh Raj	Charlie	Dog	12.50	Kennel Cough (Mild)	2025-03-28
Helen Woo	Shadow	Cat	4.50	Healthy	2025-04-15
Idris bin Saleh	Spike	Dog	24.00	Brachycephalic Airway Syn...	2025-05-02
Jasmine Kaur	Luna	Cat	6.80	Matted Fur	2025-05-30
Alice Tan	Bella	Dog	10.20	First Vaccination Series	2025-06-15
Alice Tan	Whiskers	Cat	4.20	Dermatitis improving	2025-07-01
Emily Yong	Mittens	Cat	3.00	Pre-Spay Consultation	2025-07-22

B. Indexing and Optimization

Use EXPLAIN to analyze the query plan for finding appointments before index creation.

```

EXPLAIN SELECT * FROM appointment
WHERE appointment_date BETWEEN '2025-01-01' AND '2025-06-30';

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	appointment	NULL	ALL	NULL	NULL	NULL	NULL	19	11.11	Using where

Create B-TREE index ‘idx_appt_date’ on appointment_date column to speed up date searches.

```
CREATE INDEX idx_appt_date ON Appointment(appointment_date);
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	appointment		range	idx_appt_date	idx_appt_date	4		9	100.00	Using index condition

Use EXPLAIN to analyze the query plan for finding text before index creation

```

EXPLAIN SELECT * FROM Medical_Record
WHERE diagnosis LIKE '%vaccination%' OR note LIKE '%vaccination%';

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Medical_Record	NULL	ALL	NULL	NULL	NULL	NULL	14	20.99	Using where

Create FULLTEXT index ‘idx_medical_test’ on the diagnosis, treatment, and note columns to speed up word searches.

```
CREATE FULLTEXT INDEX idx_medical_text
ON Medical_Record(diagnosis, treatment, note);

EXPLAIN SELECT * FROM Medical_Record
WHERE MATCH(diagnosis, treatment, note) AGAINST('vaccination');
```

4.0 Optimization Section

Before indexing

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	appointment	NULL	ALL	NULL	NULL	NULL	NULL	19	11.11	Using where

Before indexing, key was NULL, meaning no index was used.

After indexing

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	appointment	NULL	range	idx_appt_date	idx_appt_date	4	NULL	9	100.00	Using index condition

After creating idx_appt_date, the key column shows this index was chosen. The possible_keys column also correctly identifies our new index.

The most important change is rows dropping from 19 to 9. This shows the query is much more efficient instead of scanning the entire table (type: ALL). The database used the index to scan only 9 rows within the specified date range (type: RANGE).

Before indexing

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	Medical_Record	NULL	ALL	NULL	NULL	NULL	NULL	14	20.99	Using where

Before indexing, key was NULL, meaning no index was used.

After indexing

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	Medical_Record	NULL	fulltext	idx_medical_text	idx_medical_text	0	const	1	100.00	Using where; Ft_hints: sorted

After creating idx_medical_test, the key changed from NULL to idx_medical_text. This shows the database used our new FULLTEXT index. This also required changing the query from LIKE to MATCH... AGAINST.

The rows dropped from 14 (the whole table) to 1. Instead of a “Full Table Scan”, the database used its special text-search index (type: fulltext) to find the relevant row almost instantly. This is extremely efficient for text searches.

5.0 Team Work

Team Member	Role & Lesson Learn
LEE YIN SHEN	I'm the team leader, and my responsibility is to design the overall database structure and handle the Data Manipulation (DML) part. From these processes, I learned that foreign keys are very useful for organising information. For example, in our case, instead of repeatedly entering the full name and address of an owner like "Lee" for every animal he owns, we simply use his unique owner_id as a Foreign Key in the Pet table, and this also works for all other tables that are related to "Lee". Moreover, I also learn the foreign key constraints and realise that data must be inserted or deleted in a specific order like processing "child" data before processing "parent" data to avoid errors and keep the database links valid.
LAU YEE WEN	I was responsible for the Indexing and Optimization phase of the project. My main task was to create two types of indexes to improve query performance which is a B-Tree index for optimizing queries on numeric and date columns, and a FULLTEXT index for efficient text searching. Through this project, I learned the importance of using the EXPLAIN command to analyze how MySQL executes queries. Before indexing, the database performed a full table scan, which was slow and inefficient. After applying the B-Tree index, the query switched from a full scan to a much faster RANGE scan. I also discovered that FULLTEXT indexes require using MATCH...AGAINST instead of LIKE, which made text searching instant by locating the specific record immediately.
CHERYL CHEONG KAH VOON	In this project, I was responsible for designing and implementing the core database structure. My tasks included creating the database and tables using CREATE TABLE, modifying table structures with ALTER TABLE, and removing unnecessary tables using DROP TABLE. I ensured that all tables were logically connected through well-defined primary and foreign keys, allowing the system to maintain accurate relationships between data entities. Additionally, I applied essential constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT to enforce data consistency, prevent invalid entries, and strengthen overall data integrity. Throughout this process, I also reviewed the data model to ensure that the structure aligned with the functional requirements of the Pet Hospital System. My contribution helped establish a stable, reliable, and scalable database foundation for the system's operations.
POH LOK YEE	My role is handling the Data Query Language(DQL) tasks, where I

	focus on retrieving and processing data from the database using various SQL techniques. Throughout the work, I learned how to apply filtering, sorting, grouping and aggregation, but most importantly I gained new experience using string functions to create the Pet_Code with CONCAT and SUBSTR. I also learned to use set operations like UNION and NOT EXISTS, which I had never used before. Working with different JOINS such as INNER JOIN and LEFT JOIN taught me to think carefully about which tables and attributes to combine in order to produce meaningful and informative results. Overall these experiences strengthened my understanding of SQL and improved my ability to retrieve and present data effectively.
--	---

6.0 Conclusion

In conclusion, our team successfully designed and implemented a functional database backend for a Pet Hospital System.

In this project, one of the biggest challenges we faced is to manage the links between multiple tables. When we try to delete a record from a “parent” table like the “Pet” table, we will get an error because the data is still connected to other tables like the “Appointments” table. Then we learn that we need to delete all “child” data first, and the same rule is also applied to insert data in the correct order (first the owner, then the pet). We also found it difficult to write queries to connect multiple tables together using JOIN, because connecting owner, Pets and Vets in one command needs very careful coding to get the correct results.

For suggestion and improvement, we suggest creating a real Graphical User Interface (GUI) for this pet hospital system. This GUI will allow users to add or update information by simply clicking buttons instead of writing complex SQL code every time. We also recommend enhancing the database by adding more tables and detail. By including more specific information like medicine inventory or billing details, the system would be much closer to representing a real-world situation.

Marking Rubric

SQL Implementation (CLO1)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Backend SQL Implementation (DDL/DML/Queries)	All SQL statements execute flawlessly; covers full range – DDL, DML, SELECT, joins, subqueries, functions.	Most SQL correct; small syntax or logic issues.	Several working statements but limited variety.	Few working statements; repetitive or basic logic.	Non-functional / incomplete SQL.	
Advanced Data Logic (Filtering, Aggregation, Subquery)	Uses complex logic (nested queries, GROUP BY, HAVING, CASE, conditions) effectively.	Correct use of aggregations with minor gaps.	Some queries too simple or misapplied.	Few aggregations or logical errors.	Not attempted.	
Indexing & Optimization	Correctly creates BTREE and TEXT indexes; shows analysis before/after; interprets performance gain.	Indexes implemented; partial analysis.	Indexing attempted without analysis.	Incomplete index usage.	Not attempted.	
Transaction Control & Front-End Logic Integration	Demonstrates atomic operations (START TRANSACTION, COMMIT, ROLLBACK); explains how it fits system workflow (front-end use case e.g. booking/purchase).	Transaction logic corrects but limited integration explanation.	Partial transaction implementation.	Minimal attempt, unclear scenario.	Missing transaction logic.	
Subtotal						

Teamwork and Collaboration (CLO3)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Team Coordination & Contribution	Clear task division; balanced participation; strong collaboration evident in report and presentation; each member contributes to SQL work.	Good teamwork and participation with minor imbalance.	Some collaboration; limited task clarity.	Minimal teamwork; dominated by few members.	No teamwork evidence; incomplete group submission.	
Subtotal						

Documentation & Presentation (CLO4)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Report Documentation	Clear and complete report including intro, SQL summary, query screenshots, optimization, transaction explanation, and reflection.	Report complete with minor formatting or clarity issues.	Adequate report but lacks details or screenshots.	Weakly structured or incomplete sections.	Missing or very poor documentation.	

Slide Quality & Visual Communication	Slides are professional, organised, visually clear, and consistent with project flow.	Well-prepared slides with minor design issues.	Slides understandable but inconsistent layout.	Poorly structured or cluttered slides.	Missing or unreadable slides.	
Verbal Presentation	Excellent delivery, confident speakers, balanced time usage, fluent explanation, and effective Q&A handling.	Good delivery; some coordination issues.	Understandable but low energy; moderate Q&A.	Disorganized delivery or weak explanations.	Unable to present or answer questions.	
Subtotal						